Lecture 28: Strings in MIPS Assembly Language

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand how strings are stored in memory.
- Differentiate between `.ascii` and `.asciiz`.
- Print and read strings using MIPS syscalls.
- Traverse a string character by character.
- Find the length of a string.
- Copy and compare strings.
- Reverse a string.
- Debug string programs using the MARS simulator.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how strings are stored in memory in MIPS Assembly Language, how to use `.asciiz`, read and print strings with syscalls, and implement string operations such as length, copy, comparison, and reverse using the MARS simulator."**

In our previous lecture, we learned how arrays are stored in memory and how to process them using loops.

Today, we'll study another very important data structure: strings.

Whenever you type your name, send a message, enter a password, or search on Google, your computer is processing strings.

In MIPS Assembly Language, a string is simply an array of characters stored one after another in memory, ending with a special null character.

In today's lecture, we'll learn how to declare strings, display them, take input from the user, and perform common string operations such as finding the length, copying, comparing, and reversing strings.

Let's begin.

---

2. What is a String?

A **string** is a sequence of characters stored in contiguous memory.

Example:

HELLO

Memory representation:

Address	Value
10010000	H
10010001	E
10010002	L
10010003	L
10010004	O
10010005	\0

The last character is the **null terminator (`\0`)**, which marks the end of the string.

---

3. Declaring Strings

Using `.asciiz`

.data

message: .asciiz "Hello World"

`.asciiz` automatically appends a null character (`\0`).

Using `.ascii`

.data

message: .ascii "Hello"

`.ascii` **does not** append a null character.

Use `.ascii` only when you intentionally do not want a null terminator.

---

4. Displaying a String

To print a string:

```
.data

msg: .asciiz "Welcome to COAL"

.text
.globl main

main:

li $v0,4

la $a0,msg

syscall

li $v0,10

syscall
```

Output:

Welcome to COAL

---

5. Reading a String from the Keyboard

MIPS syscall **8** is used for string input.

Example:

.data

buffer: .space 50

.text

li $v0,8

la $a0,buffer

li $a1,50

syscall

Explanation:

- `$a0` → Address of the buffer
- `$a1` → Maximum number of characters
- `$v0 = 8` → Read String

---

6. Printing the Entered String

li $v0,4

la $a0,buffer

syscall

This displays the string entered by the user.

---

7. Traversing a String

Suppose:

name:

.asciiz "COMPUTER"

We first load the address:

la $t0,name

Then read characters one by one:

lb $t1,0($t0)

Move to the next character:

addi $t0,$t0,1

Repeat until the null character is reached.

---

8. Detecting the End of a String

Every string ends with:

'\0'

ASCII value:

0

Loop:

Loop:

lb $t1,0($t0)

beq $t1,$zero,Finish

addi $t0,$t0,1

j Loop

Finish:

---

9. Program – Find String Length (`strlen`)

Algorithm:

Length = 0

Read Character

If Character == Null

Stop

Else

Length++

Move to Next Character

MIPS Program:

.data

```
str: .asciiz "Assembly"

.text

main:

la $t0,str

li $t1,0

Loop:

lb $t2,0($t0)

beq $t2,$zero,Exit

addi $t1,$t1,1

addi $t0,$t0,1

j Loop

Exit:
```

li $v0,10

syscall

Final Result:

$t1 = 8

---

10. Program – Copy a String

Algorithm:

Read Character

Copy Character

If Character == Null

Stop

Else

Move Forward

Example:

```
.data

src: .asciiz "MIPS"

dest: .space 20

Loop:
lb $t2,0($t0)

sb $t2,0($t1)

beq $t2,$zero,Finish

addi $t0,$t0,1

addi $t1,$t1,1

j Loop
```

---

11. Program – Compare Two Strings

Logic:

Read character from String 1

Read character from String 2

If different

Strings are not equal

Else

Continue

Stop when:

- Characters differ
- Both strings reach `\0`

Mention that this is similar to the C function `strcmp()`.

---

12. Program – Reverse a String

Algorithm:

1. Find string length.

2. Point to the last character.
3. Print characters backwards.

Example:

Original

HELLO

↓

Reversed

OLLEH

---

13. ASCII Values

Characters are stored using ASCII codes.

Example:

Character	ASCII
A	65
B	66

a	97
b	98
0	48
Space	32

Use the MARS Data Segment to show that characters are actually stored as numbers.

---

14. Real-Life Analogy

Imagine a train.

H

↓

E

↓

L

↓

L

↓

O

↓

STOP

Each coach is a character.

The last coach (`\0`) tells the CPU that the string has ended.

---

15. Debugging Strings in MARS

Demonstrate:

1. Open the **Data Segment**.
2. Observe the characters stored one byte at a time.
3. Step through the loop.
4. Watch:
    - `$t0` → Current address
    - `$t2` → Current character
5. Observe the loop stopping when `$t2` becomes `0`.

---

16. Common Mistakes

Students often:

❌ Forget the null terminator.

Use:

.asciiz

instead of:

.ascii

when working with strings.

❌ Use `lw` to read characters.

Correct:

lb

because each character occupies **1 byte**.

❌ Forget to increment the pointer.

Always:

addi $t0,$t0,1

❌ Compare with `'0'` instead of the null character.

Correct:

beq $t2,$zero,Finish

🎤 Lecture Summary

Today, we learned how strings are represented and manipulated in MIPS Assembly Language.

We explored `.ascii` and `.asciiz`, used syscalls to read and print strings, traversed strings character by character, and implemented common operations such as finding the length, copying, comparing, and reversing strings. We also learned how to debug string programs using the MARS simulator.

In the next lecture, we'll begin studying **MIPS Procedures (Functions)**. We'll learn how to create reusable code using procedures, pass parameters, return values, manage the stack, and use `jal` and `jr` correctly according to MIPS calling conventions.

Thank you for watching, and I'll see you in the next lecture.
